Sahulatcart — SRS for Fazal (Stream M)

Version 1.0 · 12 October 2026 · Source plan: PLAN.md · The other developer's file: SRS-Uzair.md

This is your copy of the build spec. Parts 0 and 1 are identical in both files: the rules both developers follow, the team split, and the eight interface contracts where your work meets Uzair's. Your stream's features follow, then the shared schedule, your rows of the acceptance matrix, and the appendices you need. Read Parts 0 and 1 first.

Table of contents

- Part 0 — Rules both developers follow
- Part 1 — Team split, dependency map, interfaces between the two streams
- Part 2 — Stream M, "Meta and messaging": M1–M7 (yours)
- Part 4 — Week-by-week schedule and sync points
- Part 5 — Acceptance tests for Stream M
- Appendix A — Meta Graph API calls used, with request and response shapes
- Appendix B — New environment variables
- Appendix C — New migrations, in order

Part 0 — Rules both developers follow

0.1 What already exists (do not rebuild)

The repo is an npm workspace: `shared/` (enums and types), `backend/` (Fastify + TypeScript), `admin/` (Next.js App Router), `db/migrations/` (plain SQL, applied in filename order), `site/` (static marketing site). Read `CLAUDE.md` and `docs/PITFALLS.md` before the first change. Working today: the Roman-Urdu sales bot, negotiation engine, COD and bank-transfer orders, order slips, the merchant portal (dashboard, inbox, orders, catalog, settings, notifications, analytics), durable webhook handling, retention, backups. Live: 1 merchant, 5 products, 0 orders.

0.2 Conventions that are not optional

Rule	Where it is enforced	Why
Money is **paisa** (integer) in the backend and DB; rupees only at the reply/UI edge (`rupees()` in orchestrator, `rs()` in admin).	Everywhere	One mixed unit corrupts every total.
`merchant_id` always comes from verified auth (`resolveCtx()` in `backend/src/lib/auth.ts`), never from the request body or query.	Every admin route	Tenant isolation.
By-id routes ownership-check and return **404**, not 403, when the row belongs to another merchant.	Every by-id route	Do not confirm another tenant's ids exist.
New SQL functions end with `revoke execute on function … from public, anon, authenticated;` .	Every migration	Postgres grants EXECUTE to PUBLIC by default; `/rest/v1/rpc` would expose it.
Every `ReplySpec` kind has a Roman-Urdu `fallbackText()` .	`fallbackText()` in `backend/src/orchestrator/orchestrator.ts` ; the kinds and `TEMPLATE_ONLY_KINDS` in `backend/src/llm/types.ts`	It ships when the LLM is down or the price guard trips.
Every LLM-composed reply passes `priceGuardOk()` . New reply kinds that always say the same thing go in `TEMPLATE_ONLY_KINDS` (no LLM call).	`backend/src/orchestrator/guard.ts`	Invariant 1: the LLM never decides prices.
Admin UI: plain CSS tokens from `admin/app/globals.css` ; shared pieces from `admin/components/AppShell.tsx` (`PageHead` , `BackLink` , `EmptyState` , `StatusPill` , `humanize()`); lucide icons with `aria-hidden` ; every input has a `<label>` ; icon-only buttons have `aria-label` ; money cells `td.num` ; ids `.mono` . New colours need a token in **both** light and dark blocks.	Every admin page	Consistency and dark mode.
Admin fetches go through `admin/lib/api.ts` (`api()` , `apiJson()`), never straight to the backend. Show failures with `toast(await apiError(r), 'error')` ; never toast "Saved" without checking `r.ok` .	Every admin page	The portal proxies same-origin through `admin/app/api/[...path]/route.ts` .
No secrets in chat, logs, URLs, or the browser. `pino` redaction is configured in `backend/src/lib/logger.ts` ; add new secret field names there.	Backend	PITFALLS §1.
Tests run with `npm run test -w @app/backend` (never bare `npx vitest`). Typecheck: `npm run typecheck` and `cd admin && npm run typecheck` . Check the **exit code**, not the output.	Before every PR	PITFALLS §5.

Rule	Where it is enforced	Why
`CHANGELOG.md` gets an entry **with every PR**, recording what was tried and failed, not only the result.	Every PR	Project rule in CLAUDE.md.

0.3 Branches, PRs, deploys

- Branch names: `feat/<stream>-<id>-<slug>`, e.g. `feat/m2-embedded-signup`, `feat/p1-signup`.
- One PR per feature id (M1, P3…). Small PRs inside a feature are welcome; name them `feat/p3-push-part1`.
- A PR needs: tests green, both typechecks green, a CHANGELOG entry, and the other developer's review. Merge to `main` deploys automatically: backend on `backend/**` changes, admin on `admin/**`, and both on `shared/**` or the root `package.json`, `package-lock.json`, `tsconfig.base.json`. A docs-only PR deploys nothing.
- After a backend merge, confirm the deploy with `/healthz` (uptime resets) and `/readyz` (`db: ok`). `railway deployment list --service @app/backend` shows SUCCESS or FAILED; on FAILED read the build log before changing anything.
- Migrations: `npm run db:migrate` against the live database is run by **Fazal only**, after the PR that adds the migration is merged, and the CHANGELOG records the date. Never run a migration from a branch. Number them in order from `0009`; the sequence is fixed in Appendix C so the two streams do not collide.

0.4 Definition of done (every feature)

1. The "Done when" test of the feature passes against the **deployed** backend, not only locally.
2. Unit or flow tests exist for the new logic, in the file next to the source (`*.test.ts`).
3. The feature works in dark mode and at phone width (if it has UI).
4. CHANGELOG entry written; CLAUDE.md updated if the feature adds a rule the next person must know.
5. No new environment variable without an entry in `.env.example` and Appendix B.

0.5 Local setup for both

```
cp .env.example .env     # ask Fazal for the Supabase and Meta values; never paste them in chat
npm install
npm run build:shared
npm run dev:backend      # :8080
npm run dev:admin        # :3000
```

The replay sweeper and retention job only run with `NODE_ENV=production`, so a local backend never answers real customers even though it shares the live database. **Do not** set `NODE_ENV=production` locally. For Meta work, each developer uses the Meta app's test number and their own phone added as a test recipient (Meta allows 5).

Part 1 — Team split

1.1 Why two streams

The work splits cleanly along one line: everything that talks to **Meta** (tokens, Embedded Signup, templates, webhooks, quality, the message meter, the Meta dashboard and App Review) versus everything that is the **portal and shop operations** (signup, onboarding, alerts, order stages, opt-out, reliability, billing). The two streams share only a few interfaces (1.4), so they run in parallel and the calendar is about **9 weeks** instead of 14.

1.2 Streams and owners

Stream	Owner	Features	Weeks
M — Meta and messaging	**Fazal**	M1 per-merchant credentials and client · M2 Connect WhatsApp (Embedded Signup) · M3 connection page, quality webhooks, reconnect/disconnect · M4 templates, the window rule, fewer messages per sale · M5 message meter and cost controls · M6 platform owner console · M7 Meta dashboard and App Review	1–8
P — Portal and operations	**Uzair**	P1 self-service signup, password reset, automatic shop · P2 onboarding with readiness gates, delivery zones, COD toggle, greeting, attestations · P3 merchant alerts (Web Push + email) · P4 order stages and unpaid-order expiry · P5 opt-out enforcement and small gaps · P6 reliability: Sentry, rate limits, tests, CI · P7 billing	1–8

Fazal also owns Track A of the plan (Meta paperwork) because only the account holder can act there; M7 is the engineering half of it.

1.3 Dependency map

```
P1 signup ──► P2 onboarding ──► (needs M3 status endpoint for step 2) ──► P7 billing
                    │
M1 credentials ──► M2 connect ──► M3 status page ──► M4 templates ──► M5 meter ──► M6 console
                                                          │
P3 alerts (independent) ◄── M3 and M4 call the notify() helper from P3
P4 order stages ◄── uses sendTemplate() from M4 for the paid "order sent" button
P5 opt-out (independent)        P6 reliability (independent, but M-stream routes must adopt rate limits)
```

Nothing blocks week 1 for either developer. The first cross-dependency is in week 3 (P2 step 2 reads M3's status), and it is a read of one JSON field, so a stub is enough until M3 lands.

1.4 Interfaces between the streams (contracts; change only by agreement)

#	Provided by	Used by	Contract
IC-1	M1 (Fazal)	everything that sends	`sendText(phoneNumberId, to, body)`, `sendImage(...)`, `sendDocument(...)` in `backend/src/whatsapp/client.ts` keep their signatures. Internally they resolve the merchant's token from `phoneNumberId`. Callers never pass a token.
IC-2	M4 (Fazal)	P4 (Uzair)	`sendTemplate(phoneNumberId, to, templateName, params: string[])` returns `SendResult`. `sendButtons(phoneNumberId, to, body, buttons: {id, title}[])` for interactive replies.
IC-3	P3 (Uzair)	M3, M4, M5 (Fazal)	`notify(db, merchantId, { type, title, body, data })` in `backend/src/lib/notify.ts` writes the portal row **and** fans out to push and email per the merchant's preferences. All new notifications go through it. Until P3 lands (week 4), M1 and M3 insert `notifications` rows directly, as `handoff()` does today (`orchestrator.ts:523–530`), and switch to `notify()` at the week-4 sync.
IC-4	M3 (Fazal)	P2 (Uzair)	`GET /api/v1/admin/whatsapp` returns `{ status: 'none'|'connecting'|'connected'|'needs_attention'|'disconnected', displayPhone, displayName, qualityRating, messagingTier, templates: [{name, status}], lastTestAt }`. Until M3 lands it returns `{ status: 'none' }`.
IC-5	P1 (Uzair)	M2 (Fazal)	After P1, every signed-in portal user has exactly one `merchants` row and one owner `merchant_users` row; `resolveCtx()` returns `merchantId`. M2 relies on nothing else.
IC-6	M5 (Fazal)	P7 (Uzair)	Table `message_usage(merchant_id, phone_number_id, period_month, outbound_count, inbound_count)` is written by the client on every send and read by billing.
IC-7	M6 (Fazal)	P7, P1 (Uzair)	`isPlatformAdmin` in `MerchantCtx` becomes true for members of `platform_admins`; P7's "mark invoice paid" route checks it. A platform admin may have **no** merchant membership: `resolveCtx()` then returns `{ isPlatformAdmin: true, merchantId: null }`, and P1's `no_membership` redirect must not fire when `isPlatformAdmin` is true (otherwise the console owner gets a shop created for them).
IC-8	both	both	`backend/src/orchestrator/orchestrator.ts` is touched by M4 (message merging, buttons) and P5 (opt-out guard). Keep each change in its own small PR and merge M4's first; P5's guard sits at the top of `runOrchestrator()` and does not touch state handlers.

Part 2 — Stream M: Meta and messaging (Fazal)

M1 — Per-merchant WhatsApp credentials and the client refactor

Purpose. Today every outbound call uses one token from `.env` (`META_SYSTEM_USER_TOKEN`, read in `backend/src/whatsapp/client.ts:14,39,61`, `media.ts:9–11`, `routes/admin.ts:415`). Multi-merchant sending needs one token per merchant, stored encrypted, resolved by the number that is sending.

User story. As the platform, I send every message as the merchant who owns the number, with that merchant's own credentials, and a leaked database dump does not leak usable tokens.

Functional requirements

- FR-M1.1 A token is stored in `whatsapp_secrets` as AES-256-GCM ciphertext with a 12-byte random IV and the auth tag, under `key_id` naming the encryption key version (`'k1'`). Plaintext never touches the database, logs, or responses.
- FR-M1.2 `whatsapp_numbers.access_token_ref` points at the `whatsapp_secrets.id`. The env token is used **only** for the row whose `phone_number_id === cfg.META_DEFAULT_PHONE_NUMBER_ID` (the one pilot merchant), never for any other null ref: a null ref elsewhere means "no credentials" and the send is refused. After M2 is live and the pilot merchant has reconnected through the button, the env fallback is removed, `META_SYSTEM_USER_TOKEN` is deleted from Railway, and `db/seed.mjs` stops creating `whatsapp_numbers` rows from env.
- FR-M1.3 `sendText`, `sendImage`, `sendDocument` (and the new `sendTemplate`, `sendButtons` from M4) resolve the token from `phoneNumberId` through a cached lookup; signatures unchanged (IC-1).
- FR-M1.4 `downloadWhatsAppMedia(mediaId)` gains a `phoneNumberId` argument so it uses the right token (media ids are scoped to the WABA). Its two call sites (voice notes in `webhook.ts`, screenshots in `orchestrator.ts`) pass the conversation's number.
- FR-M1.5 Catalog sync (`POST /api/v1/admin/products/catalog-sync`) uses the merchant's token and the merchant's `waba_id`.
- FR-M1.6 A Meta error with code 190 (invalid or expired token) on any send marks the number `status='needs_attention'`, `flag_reason='token'`, and raises a `notify()` of type `system` ("WhatsApp disconnected, reconnect from Settings"). Sending stops for that number until reconnected: `tokenForNumber()` **refuses** (returns null, the send returns `{ waMessageId: null, errorCode: 'no_credentials' }`) when `status` is `disconnected`, or `needs_attention` with `flag_reason='token'`. Nothing else in the SRS gates on status, so this one function is the whole guarantee; without it, M3's disconnect would fall back to the env token and keep sending.
- FR-M1.7 The decrypt cache holds at most 500 entries for 10 minutes; a reconnect or disconnect invalidates the entry.

Data. M1 needs no new columns: `whatsapp_secrets`, `whatsapp_numbers.access_token_ref`, `token_expires_at`, `token_rotated_at`, `flag_reason` exist. Migration 0009 (shared with M2 and M3, Appendix C) adds an index `whatsapp_secrets(merchant_id, kind)`. `ciphertext`, `iv`, `auth_tag` are `bytea`; supabase-js does not accept a `Buffer`, so write them as `\\x<hex>` strings and read them back with `Buffer.from(hex.slice(2), 'hex')`.

How it works

1. `backend/src/lib/crypto.ts` (new): `encrypt(plain: string): {ciphertext, iv, authTag, keyId}` and `decrypt({ciphertext, iv, authTag, keyId}): string` using `node:crypto` `createCipheriv('aes-256-gcm', key, iv)`. The key is `TOKEN_ENCRYPTION_KEY` (32 bytes, base64) from `config.ts`, already declared (`z.string().optional()`, no format check) and unused. Make it required **when** `NODE_ENV==='production'` and validate the decoded length; `loadConfig()` calls `process.exit(1)` on a schema failure, and `backend/vitest.config.ts` supplies only `NODE_ENV`, `LOG_LEVEL` and `SUPABASE_*`, so add a dummy 32-byte key there in the same PR or every test dies at import.
2. `backend/src/whatsapp/credentials.ts` (new): - `storeToken(db, merchantId, token): Promise<secretId>` → inserts `whatsapp_secrets`, returns id. - `tokenForNumber(db, phoneNumberId): Promise<string | null>` → looks up `whatsapp_numbers` → refuses per FR-M1.6 → `access_token_ref` → `whatsapp_secrets` → `decrypt`. Env token only for the default pilot number (FR-M1.2). Cached per `phoneNumberId`. Keep today's "no credentials → `{ waMessageId: null }` without calling fetch" path (`client.ts:15–18`) for local development. - `forgetNumber(phoneNumberId)` → cache invalidation.
3. `client.ts`: replace `cfg.META_SYSTEM_USER_TOKEN` with `await tokenForNumber(db, phoneNumberId)`. `client.ts` currently has no `db`; import `getServiceClient()` from `lib/supabase.ts` as the webhook does.
4. Error path: Meta returns `{ error: { code: 190, ... } }` → call `markTokenInvalid(db, phoneNumberId)` (sets status/flag, `forgetNumber`, `notify`). Existing `SendResult` gains `errorCode?: number`.

Security. Logger redaction: `'*.access_token'` and `'*.token'` are already in the pino `redact` list (`lib/logger.ts:12`); add `'*.ciphertext'`, `'*.pin'` and the Meta response paths (`'*.response.access_token'`), since pino's `*` matches one level only. `whatsapp_secrets` is already service-role-only (RLS, migration 0002). Never return a secret row from any route.

Tests. `crypto.test.ts`: round trip, wrong key fails, tampered tag fails. `credentials.test.ts` over `FakeDb`: default pilot number → env token; other null ref → refused; ref → decrypted token; disconnected → refused; cache invalidation on `forgetNumber`; 190 error flags the number. A new `client.test.ts` with a mocked global `fetch` asserts the `Authorization` header per number. Note the flow harness (`orchestrator.flow.test.ts:32–40`) replaces `../whatsapp/client` wholesale, so nothing inside `client.ts` runs there; `client.test.ts` is where client behaviour is tested.

Done when. With two merchants in the fake DB holding different tokens, a send from each number carries that merchant's token in the `Authorization` header (assert in a mocked `fetch`), and a dump of `whatsapp_secrets` contains no plaintext.

M2 — "Connect WhatsApp" through Meta Embedded Signup

Purpose. A merchant links their own WhatsApp Business number to Sahulatcart in one click. This is the feature Meta must see on video for `whatsapp_business_messaging`, and the heart of self-service.

User story. As a merchant, I click Connect WhatsApp, log in to Facebook in Meta's popup, pick or create my WhatsApp Business account and number, and come back to the portal with my number shown as connected and a test message on my phone.

Functional requirements

- FR-M2.1 Portal page `/settings/whatsapp` (new route `admin/app/settings/whatsapp/page.tsx`) shows the connect button when status is `none` or `disconnected`; otherwise it shows M3's status view.

- FR-M2.2 The button launches Meta's Embedded Signup via the Facebook JS SDK with our `META_EMBEDDED_SIGNUP_CONFIG_ID`, `response_type: 'code'`, `sessionInfoVersion: '3'`.
- FR-M2.3 The browser sends **only** the returned `code` to `POST /api/v1/admin/whatsapp/connect`, plus the `phone_number_id` and `waba_id` Meta's `message` event reported, as **hints**. The backend never trusts the hints; it verifies them against the token (FR-M2.5).
- FR-M2.4 The backend exchanges the code for a business token (Appendix A.1), stores it (M1), and records the attempt in `whatsapp_connect_attempts` (new table) with status.
- FR-M2.5 The backend calls `debug_token` (A.2) and reads the granted WABA ids from `granular_scopes`; the hinted `waba_id` must be among them or the attempt fails with 400 `waba_not_granted`. It then lists the WABA's phone numbers (A.3) and the hinted `phone_number_id` must be in the list, else 400 `phone_not_in_waba`. If no hints were received (merchant's browser blocked the message event), the backend uses the single granted WABA and its single number; two or more → 409 `choose_number` with the list, and the page lets the merchant pick, then re-posts with `phone_number_id`.
- FR-M2.6 The backend subscribes the app to the WABA (A.4), then registers the number with a generated 6-digit PIN (A.5), stored encrypted as a second `whatsapp_secrets` row of a new kind `'pin'`. Registration outcomes:
- `{ success: true }` → fine, including for a number that was already registered with this PIN.
- **133005, two-step PIN mismatch** (the number already has a two-step PIN the merchant set earlier) → the page asks "Enter the two-step verification PIN you set for this number" and re-posts to `POST /api/v1/admin/whatsapp/retry { pin }`; the merchant's PIN replaces the generated one.
- **133006, re-verification needed** → `needs_attention`, `flag_reason='manual'`, message "Verify this number again in WhatsApp Manager (Phone numbers → Verify), then press Retry".
- **133016, registration limit** → `needs_attention`, message "Meta limits how often a number can be registered; try again in a few hours".
- Anything else → `needs_attention`, `flag_reason='manual'`, Meta's message shown verbatim. Steps A.2–A.5 run **before** the `whatsapp_numbers` row exists, so they use the raw token from the exchange, not `tokenForNumber()`; only sends after the upsert (FR-M2.7) go through the client. `debug_token`'s `expires_at` (0 = never) is written to `token_expires_at` (null for 0).
- FR-M2.7 The backend upserts `whatsapp_numbers` on `phone_number_id` with `merchant_id`, `waba_id`, `display_name` (`verified_name`), `phone_e164` (`display_phone_number` normalised), `status='connecting'`, `access_token_ref`. A `phone_number_id` already owned by **another** merchant → 409 `number_in_use`, the new token is discarded, and the attempt is logged. One merchant may own one number (decision 2); a second connect replaces the first after a confirm dialog: the old row is set `disconnected` and its secrets deleted, but the row is kept (conversations reference it with `on delete set null`).
- FR-M2.8 **Connected is proven by a handshake, not by an outbound test.** A free-form text to the merchant's personal phone would be refused by Meta (error 131047): no customer-service window is open, because that phone has never messaged the shop number, and a new WABA has no approved template yet. So after the upsert the page shows "Now send any message from your phone (+92…) to your shop number" and polls `GET /api/v1/admin/whatsapp` every 3 s. When the webhook receives an inbound from `merchants.phone` on this `phone_number_id` (which also proves the WABA subscription and routing work), the backend replies "Sahulatcart connected ✓" through `sendText` and sets `connected` only when that send returns a message id. A reply failure with 131042 (payment method missing) → `needs_attention`, `flag_reason='policy'`, message "Add a payment method to your WhatsApp Business account in Meta Business Manager, then press Retry"; 131026 → "this number cannot message that phone (check it is a WhatsApp number)". The handshake also records `last_test_at`. During development mode with Meta's **test** number, `merchants.phone` must be one of the five test recipients, or Meta refuses the reply; say so on the page when `phone_number_id` is the test number.
- FR-M2.9 Abandoning the popup (event `CANCEL`) leaves nothing written. A backend failure after the token was stored deletes the secret row and leaves status unchanged.
- FR-M2.10 The connect route is idempotent per `code`: `whatsapp_connect_attempts.code_hash` (SHA-256 of the code, unique) is written first; a second post with the same code returns the stored result (Meta codes are single-use; the second exchange would fail anyway).
- FR-M2.11 Owners and managers (`isManager()`) can connect, disconnect and resubmit templates. Spec 09 §2.3's matrix (CD-37) says only the platform admin may "connect numbers"; that row predates self-service (PLAN decision 2). The M2 PR amends spec 09 §2.3 so the matrix and the route agree.
- FR-M2.12 Every server-side Graph call carries `appsecret_proof` (HMAC-SHA256 of the token with `META_APP_SECRET`), and "Require App Secret" is switched on in the app's Advanced Settings (M7), so a leaked token alone cannot call the API.

Data (migration 0009)

```
alter type whatsapp_secret_kind add value 'pin';
create table whatsapp_connect_attempts (
  id uuid primary key default gen_random_uuid(),
  merchant_id uuid not null references merchants(id) on delete cascade,
  member_id uuid references merchant_users(id) on delete set null,
  status text not null default 'started',   -- started|token_ok|verified|subscribed|registered|awaiting_handshake|connected|failed
  code_hash text unique, waba_id text, phone_number_id text,
  error_code text, error_message text,
  created_at timestamptz not null default now(), updated_at timestamptz not null default now()
);
create index on whatsapp_connect_attempts(merchant_id, created_at desc);
alter type webhook_item_type add value 'event';   -- M3: account-level webhook fields
alter table whatsapp_numbers add column last_test_at timestamptz, add column attention_message text;
create index on whatsapp_secrets(merchant_id, kind);
```

New tables are locked for anon/authenticated by default privileges. `db/migrate.mjs` wraps each file in one transaction, and Postgres forbids **using** a new enum value in the transaction that added it, so 0009 must not insert `'pin'` or `'event'` rows itself.

API

`POST /api/v1/admin/whatsapp/connect` Request: `{ code: string, hints?: { wabaId?: string, phoneNumberId?: string }, phoneNumberId?: string }` Response 200: `{ status: 'awaiting_handshake'|'needs_attention', phoneNumberId, displayPhone, displayName, flagReason?, message? }` `POST /api/v1/admin/whatsapp/retry { pin? }` re-runs subscribe → register → (handshake wait) without a new popup. Errors: 400 `code_exchange_failed` (Meta's message included), 400 `waba_not_granted`, 400 `phone_not_in_waba`, 409 `choose_number` `{ numbers: [{id, displayPhone, verifiedName}] }`, 409 `number_in_use`, 403 for non-managers.

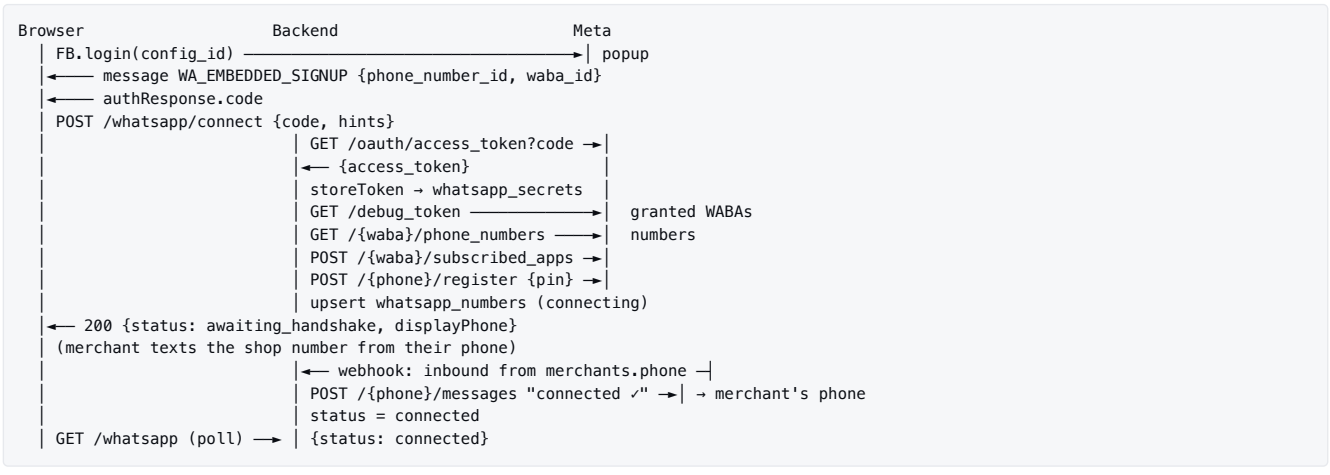
`GET /api/v1/admin/whatsapp` (IC-4; implemented in M3, stubbed here to return `status`).

`GET /api/v1/config` (exists) adds `metaAppId` and `embeddedSignupConfigId` so the page can init the SDK.

UI (admin/app/settings/whatsapp/page.tsx)

- Loads the Facebook SDK (`https://connect.facebook.net/en_US/sdk.js`) once, `FB.init({ appId, version: cfg.graphVersion, xfbml: false })` .
- Registers `window.addEventListener('message', …)` filtering `event.origin` to `https://www.facebook.com` or `https://web.facebook.com` and `data.type === 'WA_EMBEDDED_SIGNUP'` ; on `event === 'FINISH'` keeps `data.data.phone_number_id` and `waba_id` as hints.
- Button → `FB.login(cb, { config_id, response_type: 'code', override_default_response_type: true, extras: { sessionInfoVersion: '3', setup: {} } })` .
- `cb` receives `response.authResponse.code` → `api('/api/v1/admin/whatsapp/connect', { method: 'POST', body })` .
- States: idle → "Opening Meta…" → "Connecting your number…" (progress list: token, verify, subscribe, register) → "Send any message from your phone to +92… to finish" (polling, with the number shown large and a WhatsApp deep link `https://wa.me/<shop number>?text=hi`) → connected view (M3) or needs-attention view with the message from the backend, a PIN field when the error was 133005, and a **Retry** button (`POST /api/v1/admin/whatsapp/retry`).
- A plain-words explainer above the button: what the merchant needs (a Facebook login that manages their business, a phone number not already on the WhatsApp app, a payment method added in Meta), and the display-name rule (must be the real shop name).

How it works (sequence)



Security. The code is single-use and exchanged server-side with `META_APP_SECRET` ; the token never reaches the browser. The `message` listener checks `event.origin` . Hints are verified against the token. The route is rate-limited (P6) to 5 per minute per merchant.

Tests. `connect.test.ts` with a mocked `fetch` for Meta: happy path writes secret + number + attempt `awaiting_handshake` ; hint mismatch → 400 and no rows; number owned by another merchant → 409 and the stored secret is deleted; 133005 → `pin_required` ; same code twice → same result, one exchange call; handshake: an inbound from `merchants.phone` flips `connected` and sends the reply; reply 131042 → `needs_attention` with `policy` ; every Graph call carries `appsecret_proof` . UI: manual test with the Meta test WABA and a tester login (the app is in development mode; testers are added under App Roles).

Done when. A tester account connects the Meta test number end to end on the deployed portal, texts it from their phone, and receives "Sahulatcart connected ✓"; the page flips to connected without a reload; a second message gets a bot reply; `whatsapp_secrets` holds ciphertext only; the backend log contains no token.

M3 — Connection page, quality and account webhooks, reconnect and disconnect

Purpose. The merchant must always see whether their number works, and Meta's warnings (quality drop, template rejection, account restriction) must reach them. Today only the `messages` webhook field is subscribed, and `parseWebhook()` silently **drops** any change without `value.metadata.phone_number_id` (`parse.ts:52–53`), which is exactly what the account-level fields look like, so those events are invisible twice over.

Functional requirements

- FR-M3.1 `GET /api/v1/admin/whatsapp` (IC-4) returns the number's status, display phone, display name, `quality_rating` , `messaging_tier` , `verified_name_status` , the template list with statuses (M4), `lastTestAt` , and `flagReason` with a plain-words `message` .
- FR-M3.2 The webhook route handles three more fields. They carry no `metadata` ; the WABA id is `entry[].id` , and quality updates name the number by `value.display_phone_number` . Route `phone_number_quality_update` by WABA + display phone (`phone_e164`), the other two by WABA (a WABA-level event applies to every number row with that `waba_id`):
- `phone_number_quality_update` → update `quality_rating` (GREEN/YELLOW/RED → enum), `messaging_tier` from `current_limit` (TIER_250 → `unverified` , TIER_1K → `tier_1` , TIER_10K → `tier_2` , TIER_100K → `tier_3` , TIER_UNLIMITED → `tier_4`); on `event: FLAGGED` set `status='needs_attention'` , `flag_reason='quality'` ; on `UNFLAGGED` restore `connected` . Notify on every change to yellow or red.
- `account_update` → on `event: DISABLED_UPDATE` or `ACCOUNT_VIOLATION` set `needs_attention` , `flag_reason='policy'` and notify with Meta's reason; on `ACCOUNT_RESTRICTION` likewise.
- `message_template_status_update` → M4.
- FR-M3.3 `POST /api/v1/admin/whatsapp/test` sends the test message again (FR-M2.8) and updates `lastTestAt` ; it is what the Retry button and onboarding step 7 (P2) call.
- FR-M3.4 `POST /api/v1/admin/whatsapp/disconnect` : sets `status='disconnected'` , deletes the secret rows, nulls `access_token_ref` , `forgetNumber()` . It does **not** call Meta to deregister (the merchant keeps their number and WABA). Bot replies stop because `tokenForNumber()` refuses (FR-M1.6). The webhook still routes inbound by `phone_number_id` , so buyers' messages are stored and appear in the inbox, but **no auto-reply of any kind is possible**

(there is no token), and the merchant is reminded once a day through `notify()` (portal, push, email; never WhatsApp) that the bot is off and buyers are waiting.

- FR-M3.5 Reconnect is M2's button again; it replaces the secret and resets flags.
- FR-M3.6 `parseWebhook()` gains `events: { field, wabaId, entryTime, value }[]` : for every change whose `field !== 'messages'` it reads `change.field` , `entry.id` and `entry.time` (add `time` to `WebhookEntry` in `types.ts` ; `field` and `id` exist but are unused) instead of dropping it. `claimWebhookPayload()` claims each event with `item_type='event'` (new enum value, migration 0009; widen `claimEvent()` 's `'message' | 'status'` union) and event id `${field}:${wabaId}:${sha256(JSON.stringify(value))}` so a redelivery is deduped; it needs no tenant lookup, since the handler resolves rows by WABA. `processClaimed()` dispatches events to `account-events.ts` .

Data. `whatsapp_numbers` has every column needed except `last_test_at` and `attention_message` , which 0009 adds (SQL in M2). `status` is plain `text` with the comment `connecting|connected|flagged| disconnected` (0001:100) and no check constraint; this SRS uses `needs_attention` instead of `flagged` : update the column comment in 0009, the `WhatsAppNumberStatus` type in `shared/` , and `db/seed.mjs:57` .

API. As above, plus `GET /api/v1/admin/whatsapp/attempts` (last 10 connect attempts, for support).

UI. `/settings/whatsapp` connected view: a card with the display phone, verified name, a `StatusPill` for status, quality as a coloured dot with text ("Quality: green"), tier as "Up to 1,000 customers a day", a templates list (M4), buttons **Send test message**, **Reconnect**, **Disconnect** (confirm dialog, explains the bot will stop). Needs-attention view: the `attention_message` in a warning box with the one action that fixes it. Settings page gets a "WhatsApp" link row with the status pill.

How it works. Webhook: `processClaimed()` gains a branch for `events` ; each handler is a function in `backend/src/whatsapp/account-events.ts` ; all write through one `updateNumber(db, phoneNumberId | wabaId, patch)` and call `notify()` (IC-3). The Meta dashboard subscription for the four fields is done in M7; until then, the code is exercised by posting sample payloads (Meta's docs give them) to the webhook with a valid signature in tests.

Tests. `account-events.test.ts` : each field's sample payload → expected column changes and one notification; redelivered payload → no second notification. Flow: a `RED` quality update followed by a send → the send still goes (quality does not block sending; the merchant is warned).

Done when. On the deployed backend, Meta's "Test" button in the webhook configuration for each of the four fields produces the expected row changes, and the portal page reflects them within one refresh.

M4 — Templates, the 24-hour window rule, and fewer messages per sale

Purpose. Since 1 October 2026 every message past 1,000 a month per number is charged (Pakistan ≈ US$0.015, about Rs 4), **including** replies inside the 24-hour window; templates are always charged. The product must send fewer messages, send a template only when the window is closed **and** the merchant chose to pay, and still create templates because App Review requires a recording of it.

Functional requirements

Templates - FR-M4.1 At connect (M2 step after `connected`) the backend creates two **UTILITY** templates on the merchant's WABA (A.7): `payment_update` and `order_sent` , language `en` (Roman Urdu is Latin script; Meta's language code list has no Roman Urdu, and `en` is what Pakistani businesses use for it; if a reviewer rejects for "language mismatch", resubmit as `en_US` with a shorter, more English body before anything else). PLAN decision 3 said three templates; it is two, and the PLAN is corrected. Body texts and parameters are fixed in `backend/src/whatsapp/templates.ts` : - `payment_update` : "Assalam o alaikum {{1}}! Aapke order {{2}} ki payment {{3}}. {{4}}" with `{{3}}` = "verify ho gayi hai" or "verify nahi ho saki" and `{{4}}` = the slip text or the reason. - `order_sent` : "{{1}}, aapka order {{2}} bhej diya gaya hai. {{3}}" with `{{3}}` = courier/ETA line. - FR-M4.2 Each template is stored in `message_templates` (new table, columns follow spec 02: `wa_template_id` , `status` lowercase `pending|approved|rejected|paused|disabled` , `rejection_reason` , `body_json`). The `message_template_status_update` webhook maps Meta's uppercase event to the lowercase status and, on rejected or paused, writes a notification of type `template_status` (the enum value exists; the notifications page maps it to a row, today it falls back to a plain bell) with Meta's reason. Create and resubmit each write an `audit_log` row (`entity='template'` , spec 09 §5). - FR-M4.3 Portal `/settings/whatsapp` lists templates with status; **Resubmit** (managers only, spec 09 §2.3 "Templates manage") re-creates a rejected one (Meta requires a new name on resubmission if the old one is rejected within 30 days: append `_v2` , `_v3`). - FR-M4.4 `sendTemplate(phoneNumberId, to, name, params)` (IC-2) sends `type: 'template'` (A.8) and records a `messages` row with `type='template'` , `template_name` .

The window rule - FR-M4.5 `notifyBuyer()` in `payment-service.ts` **already** does most of this: it computes `windowOpen` from `conversations.window_expires_at` (:66), sends only when open (:69), and writes a `system` notification when closed (:74-85). What changes: the notification `data` gains `{ orderId, action: 'payment_update' }` , the order is marked `pending_buyer_notice='payment_update'` (new column) so the order page shows the paid button, and the wording becomes "Buyer's window closed; sending costs about Rs 4". Also: when the window is open and a PDF slip exists, `verifyPayment` sends **two** messages today (text :70, then the PDF :72); make it one by sending the document with the text as its caption, as `sendCodConfirmed` already does (`orchestrator.ts:602-607`). - FR-M4.6 `POST /api/v1/admin/orders/:id/notify-buyer { kind: 'payment_update'|'order_sent' }` sends the template (always charged), clears `pending_buyer_notice` , and returns 409 `template_not_approved` if the template is not `approved` . The order page button shows the price from `cfg.META_UTILITY_RATE_PAISA` (Appendix B; default 420 = Rs 4.20) and the free-quota state from M5. - FR-M4.7 The payment claim view shows "Free to confirm until " computed from `conversations.window_expires_at` , with a countdown; after expiry the label flips to "Window closed — confirming now sends nothing; the buyer notice becomes a paid button". When the shop is past its free 1,000 for the month (M5), the label says "Confirm until (about Rs 4; free quota used)" instead, because an in-window reply is then paid too. - FR-M4.8 A buyer message that is **only** "status", "order status", "mera order" or "order kahan hai" (whole-message regex `STATUS_RE` beside `CANCEL_RE`) answers with the latest placed order's status line without an LLM call, when the customer has at least one placed order; otherwise it falls through to the normal flow. The check sits **before** `classify()` (`orchestrator.ts:254-256`) and inside `handleEscape()` (:666) for the order-flow states. `BotIntent` already has `ask_status` but no branch handles it (:284-415); add one that gives the same answer, or the LLM path still misroutes. New `ReplySpec` kind `order_status` goes in `TEMPLATE_ONLY_KINDS` with its fallback.

Fewer messages per sale - FR-M4.9 Merge sends that are always sequential: in `handlePayment()` the bank details text (:817) and the `bank_await` reply (:818) become one message. In `sendCodConfirmed()` the PDF path is already one message (the confirmation is the document's caption, :602-607); merge the **no-PDF** path, which sends the slip text and then the caption as two (:609-610). Before changing anything, record today's message count for the scripted COD and bank flows in the CHANGELOG as the baseline. - FR-M4.10 Payment method choice uses interactive buttons: `ask_payment_method` sends `sendButtons` with `[{id:'cod', title:'Cash on Delivery'}, {id:'bank', title:'Bank Transfer'}]` (max 3 buttons, titles ≤ 20 characters), offering only the enabled methods (P2 enforces `codEnabled`). `parse.ts` puts the button **title** into `NormalizedMessage.text` (:35-36), which today's `\bcash\b` / `\bbank\b` regexes match; the **id** is only in `message.raw.interactive.button_reply.id` (`RawMessage` type, `types.ts:30-33`), so `handlePayment()` reads the id from `raw` first

and falls back to the text. - FR-M4.11 Product photos: already one `replyImage` per product ask with the first image and a price-guarded caption (`orchestrator.ts:350–353`, `398–400`). Keep; no change. - FR-M4.12 Every **successful** outbound send (a message id came back) increments `message_usage` (M5), implemented in `client.ts` so no caller can forget; failed sends are not counted.

Data (migration 0010)

```
create table message_templates (                -- names per docs/spec/02-data-model.md
  id uuid primary key default gen_random_uuid(),
  merchant_id uuid not null references merchants(id) on delete cascade,
  waba_id text not null,
  wa_template_id text,
  name text not null, language text not null default 'en', category text not null default 'utility',
  status text not null default 'pending',       -- pending|approved|rejected|paused|disabled
  rejection_reason text, body_json jsonb not null, param_count int not null default 0,
  created_at timestamptz not null default now(), updated_at timestamptz not null default now(),
  unique (merchant_id, name)
);
alter table orders add column pending_buyer_notice text;   -- null | payment_update | order_sent
```

API. `POST /api/v1/admin/whatsapp/templates/:name/resubmit`; `POST /api/v1/admin/orders/:id/notify-buyer`; the order GET includes `pendingBuyerNotice`, `windowExpiresAt`, `noticePricePaisa`.

UI. Order page: the "Buyer notice" row with the countdown or the paid button; template list on the WhatsApp page.

How it works. `templates.ts` holds the two definitions and `ensureTemplates(db, merchantId)`, called at the end of M2's connect and from Resubmit. Creation response `{id, status, category}` is stored; status flips arrive by webhook. `sendTemplate` builds `components: [{type:'body', parameters: [{type:'text', text}]}]` in parameter order.

Tests. `templates.test.ts`: creation stores rows; REJECTED webhook updates and notifies; resubmit uses a new name. Flow tests: verify 20 h after the screenshot → one free-form send; verify 30 h after → no send, `pending_buyer_notice='payment_update'`, notification written; notify-buyer route → template send with the right params; `status` keyword → status line, no LLM call (assert the fake `classify` was not called); bank flow sends one message instead of two (assert `h.sent.length`); payment choice is an interactive send and a `button_reply` with id `cod` confirms COD. The flow harness's client mock (`orchestrator.flow.test.ts:32–40`) must gain `sendTemplate` and `sendButtons` and record a `type` per send, or these tests fail with "not a function".

Done when. A template created from the portal shows APPROVED in WhatsApp Manager; the flow tests above pass; the message count for a scripted COD sale drops from today's count (measure it first and record both numbers in the CHANGELOG). **Record the two App Review videos here (M7).**

M5 — Message meter and cost controls

Purpose. Each shop must see "N of 1,000 free used this month", be warned at 80% and 100%, and choose what happens after the free quota.

Functional requirements

- FR-M5.1 `message_usage` (IC-6) is incremented on every **successful** outbound send (text, image, document, template, interactive) and every inbound message, keyed by `merchant_id`, `phone_number_id`, and `period_month` (`'2026-10'`, Asia/Karachi). Use an upsert RPC `bump_message_usage(p_merchant, p_phone, p_month, p_outbound, p_inbound)` so concurrent sends do not lose counts. The meter is an estimate: Meta counts *delivered* messages on its own calendar, and WhatsApp Manager's figure is authoritative; say so on the card.
- FR-M5.2 `GET /api/v1/admin/usage` returns `{ month, outbound, inbound, freeQuota: 1000, paidOutbound: max(0, outbound-1000), estimatedCostPaisa, afterQuota: 'continue'|'handoff' }`.
- FR-M5.3 Dashboard card: "Messages this month: 412 of 1,000 free" with a bar; at ≥80% the bar turns amber and a notification is written once per month; at 100% once more. The copy says what happens next.
- FR-M5.4 Setting `settings.afterFreeQuota` (`'continue'` default | `'handoff'`; add the rule to `invalidSettingsField()` in `lib/validation.ts`, which today checks only three keys): when `'handoff'` and `outbound ≥ 1000`, the orchestrator hands new conversations to the merchant (`handoff()` with reply kind `handoff`: one free-form message, which is itself one paid message, about Rs 4, and the settings copy says so) instead of selling; existing order-flow states finish their flow (a buyer mid-checkout is never dropped).
- FR-M5.5 Human replies from the inbox count and are shown in the same meter; the inbox composer shows "paid" next to Send when the quota is used.
- FR-M5.6 The utility rate is config (`META_UTILITY_RATE_PAISA`), shown wherever a price is shown, with the text "Meta's rate; check the current card".

Data (migration 0011). `message_usage` table with `unique (merchant_id, phone_number_id, period_month)`; the `bump_message_usage` function with the revoke line.

Tests. `client.test.ts` (mocked `fetch`, FakeDb twin of `bump_message_usage`): 50 concurrent successful sends produce a count of 50 and a failed send adds nothing; the 80% notification is written once; flow test: `afterFreeQuota='handoff'` at 1000 hands off a new chat but lets an in-progress checkout finish.

Done when. The dashboard of the pilot merchant shows a count that matches, for the same month, the number of outbound `messages` rows with a `wa_message_id` (not every send writes a `messages` row today; the comparison is a sanity check, and any gap is listed in the CHANGELOG).

M6 — Platform owner console

Purpose. One private page where the platform owner sees every shop and can suspend one. Today `platform_admins`, the JWT claim and the RLS policy exist, but `jwtCtx()` pins `isPlatformAdmin: false` (`lib/auth.ts:73`).

Functional requirements

- FR-M6.1 `jwtCtx()` looks up `platform_admins` by `auth_user_id` and sets `isPlatformAdmin` true when a row exists and is active, **also when the user has no merchant membership** (today `jwtCtx()` returns null without one, `lib/auth.ts:72`): the ctx is then `{ isPlatformAdmin: true, merchantId: null, … }`, and every merchant route still 401s it (IC-7; P1's `no_membership` redirect must check this flag first). `adminCtx()` already sets `isPlatformAdmin: true` for the shared `ADMIN_API_TOKEN` (:56-59); that stays for the pilot and is removed when M6 ships.

- FR-M6.2 Routes under `/api/v1/platform/*`, all requiring `isPlatformAdmin`. They are registered **inside** the `adminRoutes` plugin (`routes/admin.ts:28–34`): its auth `preHandler` is encapsulated there, so a separate plugin would get no auth at all. Routes: `GET /merchants` (id, business name, owner email, status, plan, connection status, last inbound at, orders this month, usage this month), `POST /merchants/:id/suspend {reason}`, `POST /merchants/:id/unsuspend`, `GET /merchants/:id/attempts` (connect attempts).
- FR-M6.3 `merchants.status='suspended'` is enforced: the webhook stores inbound messages but the orchestrator sends nothing (`runOrchestrator()` must add `status` to its merchants select, `orchestrator.ts:185–188`, and return before any reply); every admin route for that merchant returns 403 `merchant_suspended` except `GET /me` and `GET /settings` so the portal can show the suspension notice.
- FR-M6.4 Page `/platform` in the admin, visible only when `GET /me` returns `isPlatformAdmin`; a table with search, a suspend/unsuspend action with a reason dialog, and a link to view the shop's connection attempts. No impersonation.
- FR-M6.5 Suspending writes an `audit_log` row (entity `merchant`, action `suspend`, the reason).

Data. None new; `merchants.status` exists. Seed Fazal's auth user into `platform_admins` by a one-off SQL in the CHANGELOG, never in a migration.

Tests. Cross-tenant: a platform admin lists all merchants; a normal owner gets 403 on `/platform/merchants`; a suspended merchant's inbound message creates no outbound send.

Done when. The console lists the pilot merchants with live connection status and a suspension stops the bot within one message.

M7 — Meta dashboard configuration and App Review

Purpose. The engineering half of Track A: configure the app so M2–M4 can run, then produce the review material.

Tasks (in order; each depends on the owner's paperwork, Track A1–A3, being done)

1. **App setup.** Developers dashboard → Create App → type Other → Business → attach to the verified portfolio. Add products **WhatsApp** and **Facebook Login for Business**. Settings → Basic: icon `site/assets/logo-1024.png`, category Business and Pages, Privacy `https://www.sahulatcart.com/privacy.html`, Terms `/terms.html`, Data Deletion `/data-deletion.html`, App Domains `sahulatcart.com`.
2. **Embedded Signup configuration.** Facebook Login for Business → Configurations → Create → "WhatsApp Embedded Signup" type, assets: WhatsApp Business accounts, permissions `whatsapp_business_management`, `whatsapp_business_messaging`. Copy the **Configuration ID** into Railway as `META_EMBEDDED_SIGNUP_CONFIG_ID`. Add both developers and the pilot merchants' Facebook accounts under **App Roles** → **Testers** (required while the app is in development mode). Tell pilot merchants to verify their own business in Meta Business Manager: an unverified business is capped at 250 messages a day and its display name stays "pending", which the connect page should say in plain words.
3. **Webhook.** WhatsApp → Configuration → Callback URL `https://appbackend-production-dae8.up.railway.app/api/v1/webhook/whatsapp`, Verify token = the current `META_WEBHOOK_VERIFY_TOKEN`. Subscribe fields: `messages`, `message_template_status_update`, `phone_number_quality_update`, `account_update`. Press Test on each and confirm the backend log shows a claim.
4. **Graph version.** Set `META_GRAPH_API_VERSION` on Railway to the newest version listed in the dashboard's API version drop-down (`.env.example` already says `v25.0`; `config.ts` still defaults to `v21.0`); note its retirement date in CLAUDE.md. Make the variable required **in production** (same pattern as `TOKEN_ENCRYPTION_KEY`, M1) and add a dummy to `backend/vitest.config.ts`, so a stale default can never silently return and the test suite still boots. Also switch on **Require App Secret** in the app's Advanced Settings (FR-M2.12).
5. **Tech Provider enrolment** (owner clicks; Fazal beside them).
6. **App Review submission.** Permissions and Features → request Advanced Access for exactly `whatsapp_business_messaging` and `whatsapp_business_management`. For each, a written description (template text in PLAN.md A6) and one uncut screen recording: - messaging: tester merchant's phone receives a buyer text → bot reply appears in the WhatsApp app on screen → portal inbox shows the same thread. - management: `/settings/whatsapp` → Resubmit a template → WhatsApp Manager shows it pending/approved. Record at 1080p with the portal zoomed so text is legible. Read the approval criteria shown in the dashboard before submitting; they changed in May 2026.
7. **Approval day runbook** (PLAN.md): Live mode, confirm Advanced Access, set `SIGNUP_OPEN=true`, one real merchant connects.

Done when. Step 3's four test events reach the backend; step 6 is submitted and the submission ids are recorded in the CHANGELOG.

Part 4 — Week-by-week schedule and sync points

Week	Fazal (M)	Uzair (P)	Sync
1	M1 crypto + credentials + client refactor; M7 steps 1–4 as soon as Track A1–A3 allow	P1 signup, confirm, reset, bootstrap RPC	Mon: agree IC-1..IC-8 line by line. Fri: both PRs reviewed.
2	M2 connect route + page (mocked Meta first, then the test WABA)	P2 readiness endpoint, delivery-zone CRUD, wizard shell	IC-4 stub agreed (`{status:'none'}`).
3	M3 status page, account/quality webhooks, disconnect	P2 wizard panels 1–7, COD enforcement, greeting	P2 step 2 embeds M2's component: pair for an hour.
4	M4 templates, window rule, message merging, buttons; **record videos, submit App Review** (M7 step 6)	P3 `notify()` , push, email, settings page	M3/M4 switch their notifications to `notify()` (IC-3).
5	M5 meter, usage endpoint, dashboard card, after-quota mode	P4 order stages, dispatch notice, unpaid expiry job	P4's dispatch uses `sendTemplate` (IC-2).
6	M6 platform console; suspension enforcement	P5 opt-out, STOP, small gaps; P6 Sentry, rate limits, lockout fix	IC-8: M4's orchestrator PR merged before P5's.
7	Pilot shops onboarded as testers; M7 approval-day prep; fix-up	P6 tests + CI; P7 schema, trial, invoices	IC-6/IC-7: billing reads the meter; console gets Invoices tab.
8	Pilot support; Meta review follow-ups	P7 billing page, enforcement, jobs	Pilot stop rules reviewed daily (PLAN.md B6).
9–10	Pilot (both). Zero wrong prices, zero duplicate orders, zero cross-shop leaks, or onboarding pauses.		Weekly: measure messages per sale and cost per shop.

Daily: a 10-minute call. Every PR reviewed by the other developer within one working day. The CHANGELOG is the shared log; disagreements on an interface are settled by editing Part 1.4 in a PR.

Part 5 — Acceptance tests for Stream M

#	Scenario	Expected	Feature
T4	Tester connects the Meta test number	Secret stored encrypted, number `connected`, test text received	M2
T5	Second tester tries the same number	409, first merchant unaffected	M2
T6	Payment method missing on the WABA	`needs_attention` with the "add a payment method" message	M2
T7	Quality webhook RED	Status `needs_attention`, notification, page shows it	M3
T8	Disconnect, then message the number	Inbound stored, no bot reply, merchant told	M3
T9	Template REJECTED webhook	Row updated, notification with reason, Resubmit works with `_v2`	M4
T10	Verify payment 20 h after screenshot	Free-form message sent, counted in meter	M4
T11	Verify payment 30 h after	Nothing sent; order shows the Rs 4 button; pressing it sends the template	M4
T12	Buyer sends "status"	Status line, no LLM call	M4
T13	COD sale, scripted	Fewer messages than the recorded baseline; payment choice is buttons	M4
T14	1,000 outbound reached with `afterFreeQuota=handoff`	New chat handed off, in-progress checkout completes	M5
T15	Platform admin suspends a shop	Next inbound gets no reply; owner's portal shows the notice	M6

Appendix A — Meta Graph API calls used

All calls use `https://graph.facebook.com/{META_GRAPH_API_VERSION}/…`. "Business token" is the merchant's token from M1. Error bodies are `{ error: { message, type, code, error_subcode, fbtrace_id } }`.

#	Call	Auth	Request
A.1	`GET /oauth/access_token?client_id={META_APP_ID}&client_secret={META_APP_SECRET}&code={code}`	app	—
A.2	`GET /debug_token?input_token={business token}`	`access_token={META_APP_ID}|{META_APP_SECRET}`	—
A.3	`GET /{waba_id}/phone_numbers?fields=id,display_phone_number,verified_name,quality_rating,code_verification_status,platform_type`	business token	—
A.4	`POST /{waba_id}/subscribed_apps`	business token	—
A.5	`POST /{phone_number_id}/register`	business token	`{ messaging_pr 'whatsapp', pi`
A.6	`POST /{phone_number_id}/messages`	business token	`{ messaging_prod to, type:'text } }`
A.7	`POST /{waba_id}/message_templates`	business token	`{ name, langua category:'UTIL components:[{ text, example: [[…]] } }] }`
A.8	`POST /{phone_number_id}/messages` (template)	business token	`{ messaging_prod to, type:'temp template:{ nam code:'en' }, c type:'body', p type:'text', t`
A.9	`POST /{phone_number_id}/messages` (buttons)	business token	`{ messaging_prod to, type:'inte interactive:{ body:{ text }, buttons:[{ typ reply:{ id, ti` (max 3, titles ≤ 20
A.10	`GET /{media_id}` then `GET {url}`	business token	—
A.11	`GET /{waba_id}/message_templates?fields=name,status,language,category,id`	business token	—

Webhook payloads (field → `entry[].changes[].value`): - `messages`: as parsed today. - `message_template_status_update`: `{ event: 'APPROVED'|'REJECTED'|'PAUSED'|'DISABLED', message_template_id, message_template_name, message_template_language, reason }`. - `phone_number_quality_update`: `{ display_phone_number, event: 'FLAGGED'|'UNFLAGGED'|'ONBOARDING'|'UPGRADE'|'DOWNGRADE', current_limit: 'TIER_250'|'TIER_1K'|'TIER_10K'|'TIER_100K'|'TIER_UNLIMITED' }`. - `account_update`: `{ phone_number, event: 'VERIFIED_ACCOUNT'|'DISABLED_UPDATE'|'ACCOUNT_VIOLATION'|'ACCOUNT_RESTRICTION'|'PARTNER_REMOVED', ban_info?: { waba_ban_state, waba_ban_date }, restriction_info?[] }`.

Embedded Signup (browser): `FB.login(cb, { config_id, response_type: 'code', override_default_response_type: true, extras: { sessionInfoVersion: '3', setup: {} } })`; the `message` event `data = { type: 'WA_EMBEDDED_SIGNUP', event: 'FINISH'|'CANCEL'|'ERROR', data: { phone_number_id, waba_id, business_id } }`.

Meta's field names and version-specific behaviour must be re-checked against the developer docs when M2 starts; the dashboard's API version drop-down shows which versions are live.

Appendix B — New environment variables

Variable	Service	Set by	Purpose
`TOKEN_ENCRYPTION_KEY`	backend	Fazal (exists in config.ts and railway.ts, unused)	32-byte base64 key for M1; required in production; dummy in vitest.config.ts
`META_APP_ID`	backend, admin (public via `/config`)	Fazal (exists in config.ts:21 and railway.ts; only the `/config` exposure is new)	Embedded Signup and token exchange
`META_EMBEDDED_SIGNUP_CONFIG_ID`	backend (public via `/config`)	Fazal	Embedded Signup configuration
`META_GRAPH_API_VERSION`	backend	Fazal	made required (no default) in M7
`META_UTILITY_RATE_PAISA`	backend	Fazal	shown wherever a paid message is offered; default 420
`SIGNUP_OPEN / SIGNUP_ALLOWLIST`	backend	Fazal	P1 gate
`VAPID_PUBLIC_KEY / VAPID_PRIVATE_KEY / VAPID_SUBJECT`	backend (public key via `/config`)	Uzair generates, Fazal sets	Web Push
`RESEND_API_KEY / ALERT_EMAIL_FROM`	backend	Fazal	email alerts
`SENTRY_DSN` (backend), `NEXT_PUBLIC_SENTRY_DSN` (admin)	both	Fazal	error alerting
`PLAN_BASIC_PRICE_PAISA / PLAN_PRO_PRICE_PAISA / PLATFORM_BANK_DETAILS`	backend	owner	billing
removed: `META_SYSTEM_USER_TOKEN`	backend	Fazal, after the pilot merchant reconnects	per-merchant tokens only

Every variable is added to `.env.example` with a comment, and declared as `preserve()` in `.railway/railway.ts` where the services need it, in the same PR that reads it.

Appendix C — New migrations, in order

File	Stream	Contents
`0009_whatsapp_connect.sql`	M	`whatsapp_secret_kind + 'pin'`; `webhook_item_type + 'event'`; `whatsapp_connect_attempts` (with `code_hash`); `whatsapp_numbers.last_test_at`, `.attention_message`, status comment updated; index on `whatsapp_secrets(merchant_id, kind)`. Must not use the new enum values itself (one transaction)
`0010_message_templates.sql`	M	`message_templates` (spec 02 column names, lowercase statuses); `orders.pending_buyer_notice`
`0011_message_usage.sql`	M	`message_usage`; `bump_message_usage()` with revoke
`0012_self_signup.sql`	P	`create_merchant_for_user()` with revoke; `merchants.settings` default `botEnabled:false`
`0013_onboarding.sql`	P	`merchants.logo_url`
`0014_push_subscriptions.sql`	P	`push_subscriptions`
`0015_order_stages.sql`	P	`orders.courier`, `.tracking_ref`, `.eta_text`
`0016_products_soft_delete.sql`	P	`products.deleted_at`
`0017_billing.sql`	P	`subscriptions`, `invoices`

Each migration is applied to the live database by Fazal after its PR merges; `fake-db.ts` gets a JS twin of every new RPC in the same PR.